An introduction to PorePy

DFN Workshop

Uppsala June 2026

Eirik Keilegavlen

PMG

PorePy

UNIVERSITAS BERGENSIS

Scope of PorePy

- Goal: Prototyping tool for multiphysics simulations in fractured porous media
- Written in Python
- Distinguishing feature: Fractures and their intersections are explicitly represented in the grid
- Financing mainly through projects on geothermal energy
- Two main strands of multiphysics models:
  - Thermoporomechanics (single phase) + fracture deformation
  - Non-isothermal multiphase multicomponent transport

  ➔Subsets of physics are also supported

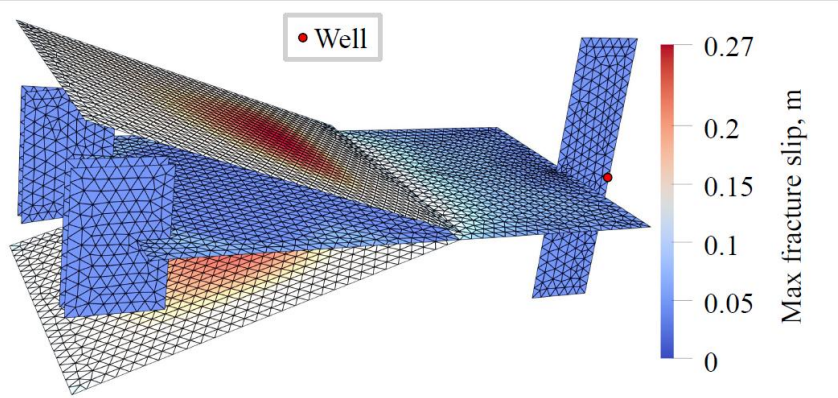
  ➔Initial efforts towards merging the strands are under way

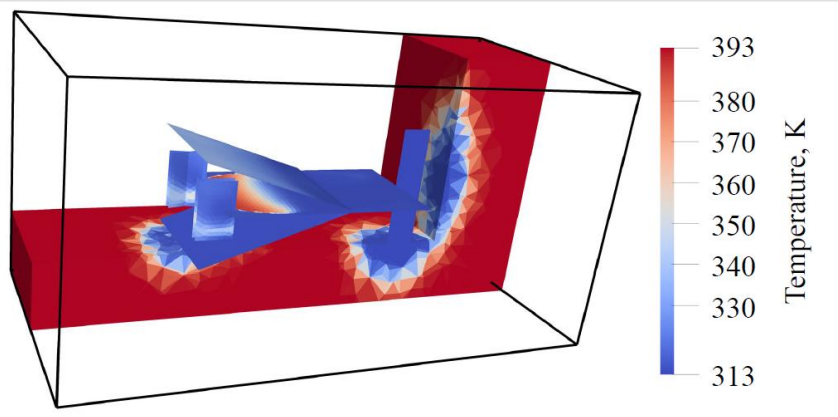
Representation of physical processes

- Governing equations are formulated in terms of partial differential equations
  - The code strives to faithfully preserve the information encoded in the equations
- Modeling approach:
  - Conservation principles (mass, energy, momentum) are strictly enforced
  - Supplemented by constitutive laws (anything goes, numerics permitting)
- Equations are written on integral form, discretized by finite volume methods
- The user will most often not interact directly with spatial discretizations

Simulation highlights

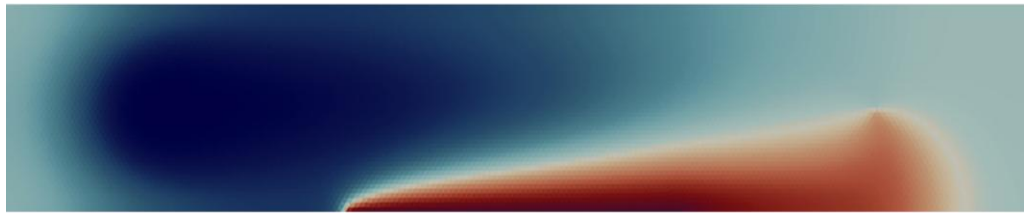
Fracture deformation under thermohydromechanical forces



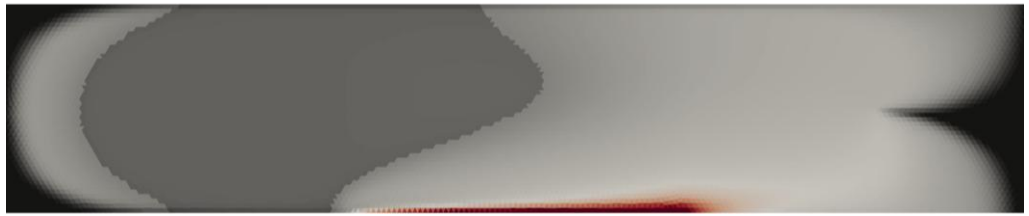


Zabegaev et al, CMAME 2025

Multiphase flow with strong thermal gradients



Temperature



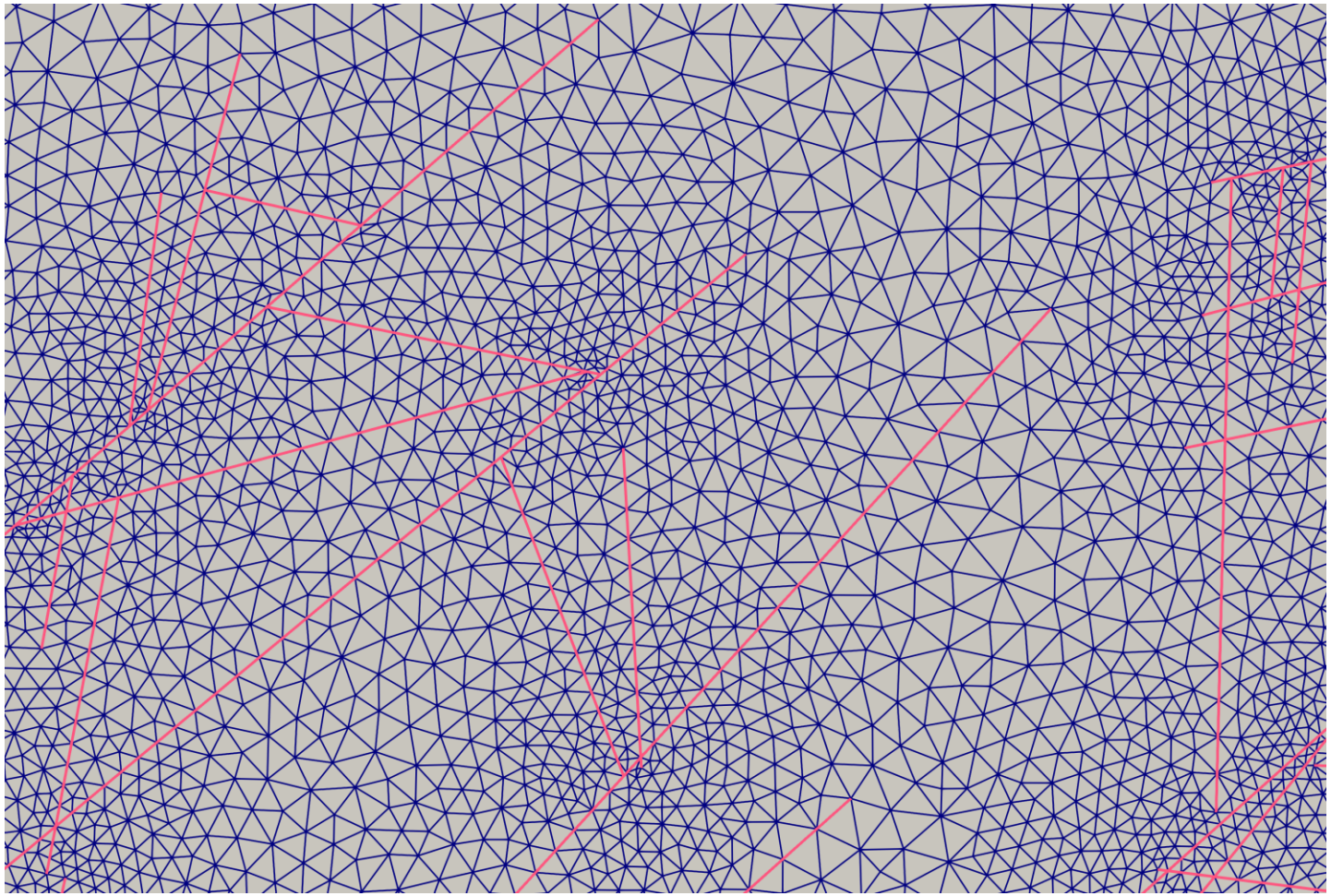
Gas saturation

Lipovac et al, arXiv: 2512.04205

How to interact with the code

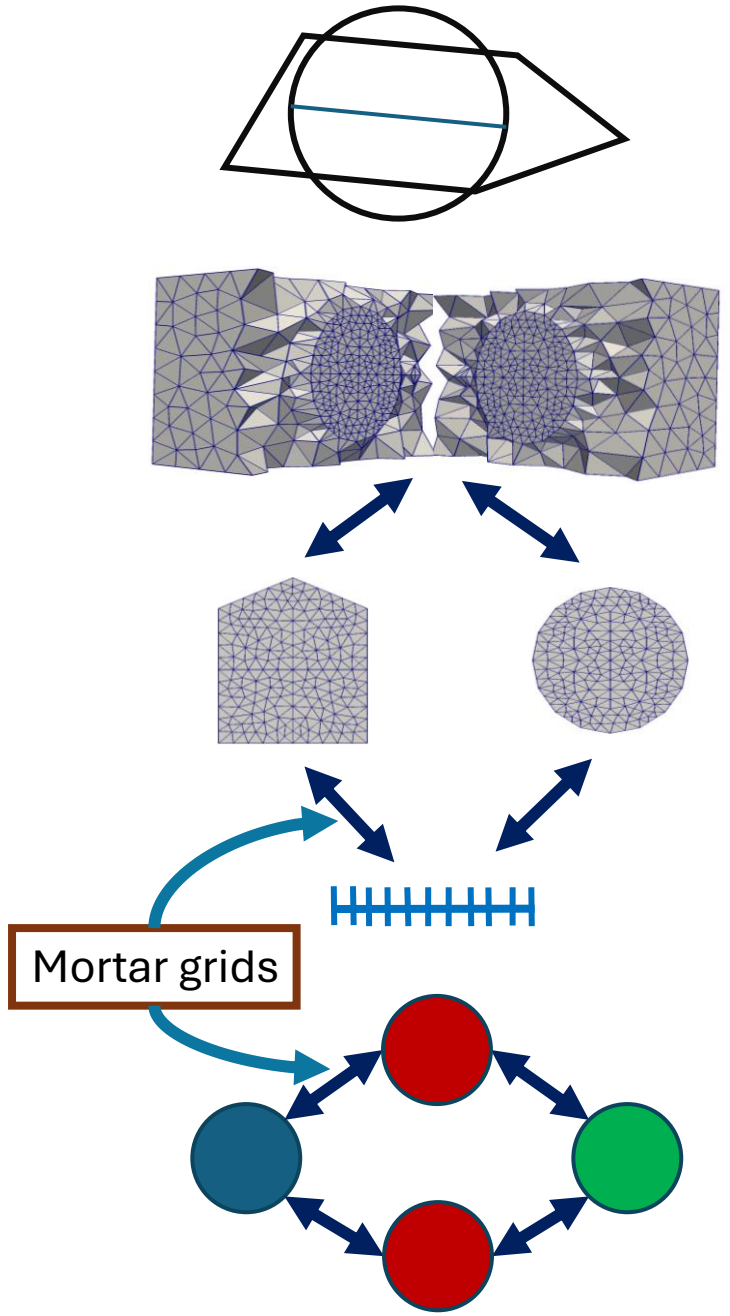
- Installation:
  - To try the code: Use GitHub codespaces
  - Full installation: The recommended approach is to use Docker
  - Installation on Linux is relatively straightforward
  - Mac and Windows may or may not work
- A simulation is set up by coding – there is no GUI
- The code ships with read-to-run examples for central physical processes
- Fair warning: The learning curve may at times be steep

Conforming mesh: 2d example



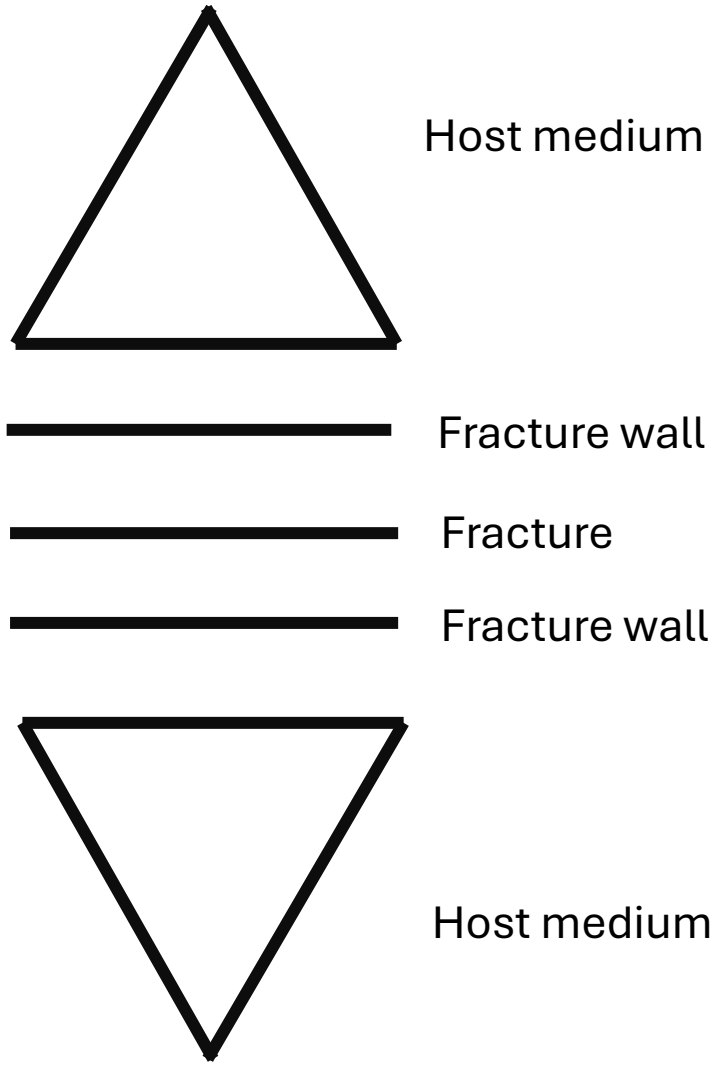
Mixed-dimensional meshes

- Mesh is constrained to geometric objects of all dimensions
- Matrix, fractures and intersections are represented by individual grids
- Variables and equations can be imposed on any combination of grids
- Mesh construction:
  - Is outsourced to Gmsh
  - See tutorial for mesh size control



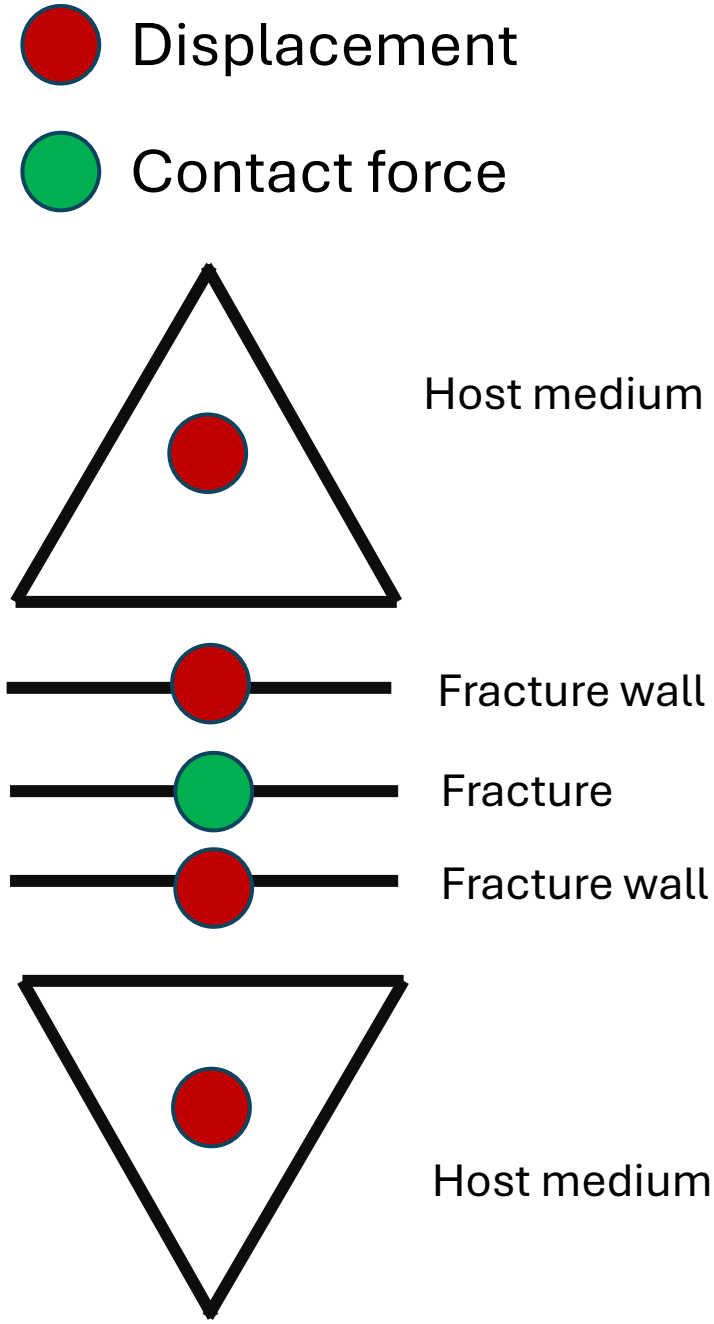
Modeling example: Fracture deformation

- Deformation is characterized as a displacement jump (normal and tangential) between the fracture walls
- Contact force between the fracture walls – Newton's third law
- Frictional contact problem:
  - No contact
  - Sticking: Friction force is not overcome
  - Sliding: Friction force is insufficient to withstand tangential traction



Modeling example: Fracture deformation

- Deformation is characterized as a displacement jump (normal and tangential) between the fracture walls
- Contact force between the fracture walls – Newton's third law
- Frictional contact problem:
  - No contact
  - Sticking: Friction force is not overcome
  - Sliding: Friction force is insufficient to withstand tangential traction



Programming style

- The code is (almost) exclusively Python
- Multiphysics models:
  - Can make use of a library of constitutive
  - Make heavy use of Python mixins (powerful, but easy to make mistakes)
- Best practice when setting up a simulation:
  - If possible, start from a working example
  - Be careful with inheritance

Organization

- Mainly developed at the University of Bergen (Norway)
  - Development team consists of PhD students and permanent researchers
  - Development is financed through research projects
- External users are supported as well as we can, resources permitting

- Hosted on GitHub (www.github.com/pmgbergen/porepy)
  - Extensions go through automatic testing and code review
- Questions or comments to the code: Please make issues or questions through GitHub

Release policy

Guidelines that we try to follow:

- Aim at two yearly releases (late spring, late fall) – git branch ‘main’
- Make a reasonable (but resource bound) effort to stabilize the code prior to releases
- Mark functionality for deprecation 1-2 releases before it is deleted

Disclamer: PorePy is in an alpha stage

- Code may be altered, moved, renamed or deleted at short notice
- Let us know if changes cause problems

Modular design of multiphysics models

- Constitutive laws are implemented with emphasis on modularity and flexibility
- Individual terms/factors in equations can be altered surgically

```python
def stress(self, subdomains: list[pp.Grid]) -> pp.ad.Operator:
    """Thermo-poromechanical stress operator."""
    stress = (
        self.mechanical_stress(subdomains)
        + self.pressure_stress(subdomains)
        + self.thermal_stress(subdomains)
    )
    return stress

def pressure_stress(self, subdomains: list[pp.Grid]) -> pp.ad.Operator:
    """Pressure contribution to poromechanical stress tensor."""
    for sd in subdomains:
        if sd.dim != self.nd:  # Stress is only defined in the matrix.
            raise ValueError("Subdomain must be of dimension nd.")

    discr = pp.ad.BiotAd(self.stress_keyword, subdomains)
    # The stress is the grad_p discretization operator times pressure perturbati
    stress: pp.ad.Operator = discr.grad_p @ (
        self.pressure(subdomains) - self.reference_pressure(subdomains)
    )
    return stress
```

Solving linear and non-linear systems

- Non-linear systems are solved with Newton's method supported by line-search
- Linear systems are by default solved with direct solvers
  - Robust, but prohibitively expensive for large systems
  - Iterative solvers (PETSc) are available through a separate repository/Docker image
- Equations are solved fully coupled (accurate, but prone to convergence issues)

Weak points

- Computational cost (CPU and memory)
  - Python..
    - Partly alleviated by iterative solvers
    - Acceleration of slow parts of the code is in the works
  - Meshes that conform to fractures have many cells
- Nonlinear systems are hard to solve (see above slide)
- Stabilizing complex simulations requires trial and error

Convergence of non-linear problems

- Multiphysics simulations involve vastly different scales
  - Results in poorly conditioned linear systems -> convergence issues
  - Partial solution: Use PorePy's system of Unit scalings (see tutorials)
  - Inherent problem: Fracture aperture is measured in mm, domain size in km
- (Newton) convergence criteria should be set with care
  - Some trial and error may be needed

MaPSI project (2021-2026). This project has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (grant agreement No 101002507).